

Sahayak

An offline AI health worker in every pocket

Camera + voice + a language model running **fully on the phone** — no internet, no per-query cost, no health data leaving the device. Built for India's ~1 million ASHA community health workers and the villages they serve at zero bars.

THIS IS NOT JUST AN IDEA

A working prototype ships with this submission. Installable APK, real on-device AI inference (llama.cpp), camera-to-voice chain working end-to-end. Our demo runs with airplane mode ON.

The last mile of healthcare has no signal

India's rural healthcare runs on ~1 million ASHA workers — community health workers who walk village to village with paper forms, no diagnostic support, and often no network coverage at all. The patients they serve, many with low literacy, misread medicine labels, miss danger signs, and travel hours to a Primary Health Centre for questions a trained assistant could answer on the spot.

Why cloud AI cannot fix this

- **No connectivity.** Village visits happen at zero bars. A chatbot that needs the internet is useless exactly where it is needed most.
- **No budget.** Per-query API costs do not scale to a million workers making daily visits.
- **No trust.** Health conversations are private. Data leaving the phone is an adoption barrier — and a real risk.

The phone already in every pocket, made clinical

Sahayak uses the three things every Android phone already has — a camera, a microphone, and enough compute for a small language model — to put an offline AI assistant in the field. Everything runs on-device.

MODULE	WHAT IT DOES	STATUS
Medicine Lens	Point the camera at any medicine strip or prescription → the phone reads it (on-device OCR) and speaks what it is, the printed dosage, and warnings — English, Hindi, or Telugu.	✓ BUILT — IN PROTOTYPE
Voice Triage	The patient describes symptoms by voice → on-device AI returns a cautious screening result: urgency (green/amber/red), practical advice, and a ready-to-read referral note for the PHC.	✓ BUILT — IN PROTOTYPE
Model Manager + Prompt Lab	Hot-swap any open-source model at runtime; tune and test prompts on the phone itself, with live tokens/sec.	✓ BUILT — IN PROTOTYPE
Camera screening	Reference-band mid-upper-arm check for child malnutrition screening.	30-HOUR BUILD
Auto-filled paperwork	The visit transcript auto-populates the government health form; PDF export; syncs when network appears.	30-HOUR BUILD

We de-risked the idea before asking you to bet on it

We didn't just write this deck — we built the risky parts first. The repository and an installable APK are linked on the cover page.

Working today, verifiable by anyone

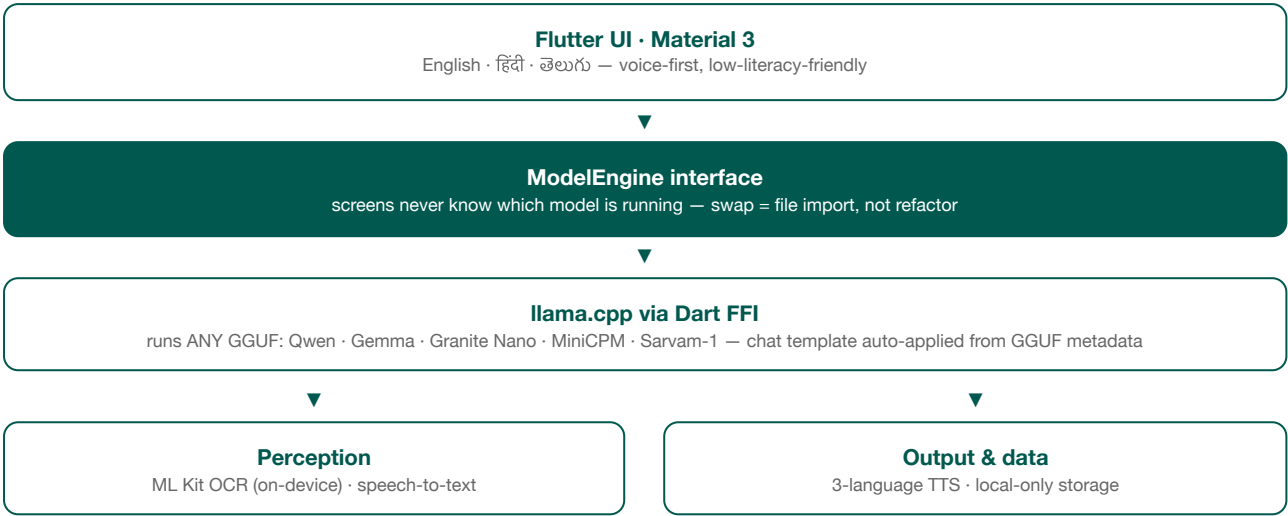
- **Full chain, end to end:** camera → on-device OCR (ML Kit) → local LLM (llama.cpp) → spoken answer (TTS). Voice input → structured triage JSON → urgency verdict → spoken advice.
- **Real on-device inference:** any open-source GGUF model — Qwen3 0.6B, Gemma, IBM Granite Nano, Sarvam-1 — imports at runtime and runs locally. **There is no API key anywhere in the codebase.**
- **Measured, not promised:** the in-app Prompt Lab shows live tokens-per-second for every run.
- **Installable now:** 62 MB arm64 release APK on the GitHub release, rebuilt by CI on every version tag.

Transparency by design: this prototype is our pre-event feasibility spike. Our last commit before the battle will be tagged **pre-event**, so judges can diff exactly what was created during the 30 hours. We want the before/after to be checkable.

What the prototype proves

RISK MOST TEAMS DISCOVER AT HOUR 20	OUR STATUS
Can a quantized LLM actually load and run on an Android phone, offline?	Proven — llama.cpp via FFI, GGUF import at runtime
Can OCR read real medicine packaging reliably enough to be useful?	Proven — on-device ML Kit, editable text fallback in UI
Does voice in / voice out work in Indian languages?	Working — platform STT/TTS wired for en-IN / hi-IN / te-IN; Whisper-class STT is a battle upgrade
Will the app build and ship?	Proven — release APK builds clean in CI and locally

Model-agnostic, low-end-first architecture



Every box runs on the phone. Nothing above requires a network connection.

One APK from ₹8,000 phones to the iQOO 15

ASHA workers don't carry flagships — so inference is CPU-first (runs on any Android), with GPU/NPU acceleration as a bonus where the silicon exists. The app picks the model tier for the device:

DEVICE CLASS	MODEL TIER (Q4 QUANTIZED)
3–4 GB RAM (entry phones)	0.3–1B — Qwen3 0.6B, Granite Nano, Gemma 1B
6–8 GB RAM	1–2B — Gemma 2B, Sarvam-1 2B (open model built for 10 Indic languages)
Flagship — iQOO 15	2–4B models; Snapdragon GPU/NPU acceleration (ONNX Runtime QNN path) targeted during the battle

Why on-device AI is necessary here, not a gimmick: offline villages + private health data means local inference is the *only* architecture that works. Cloud isn't a cheaper alternative we skipped — it's a disqualifying dependency we removed.

The details most teams will miss

- **Phone-only build windows are productive for us, by design.** The in-app **Prompt Lab** lets us tune prompts, swap models, and measure tokens/sec on the phone itself. Red Light hours become our tuning and field-testing hours, not dead time.
- **Office Kit in the loop, not bolted on.** Green Light development runs through phone–laptop bridging: file transfer for GGUF models, clipboard for prompts, screen mirror for live debugging — the exact workflow this format is designed to showcase.
- **A demo that cannot be faked.** Airplane mode ON, in front of the judges: scan a real medicine strip — the phone speaks in Telugu. Describe symptoms — urgency verdict and referral note, out loud. ~90 seconds, no Wi-Fi, no excuses.
- **Native feel.** Material 3, big-tap targets, voice-first flows designed for the actual end user — a health worker in the sun, not a developer at a desk.

The 30-hour battle plan

HOURS	WORK
0 – 6	NPU acceleration on the iQOO 15; Sarvam-1 2B as default Indic model; field-tune triage prompts on-device via Prompt Lab
6 – 16	ASHA visit workflow: patient record → auto-filled government health form → PDF export; offline-first sync stub
16 – 24	Camera screening module (reference-band malnutrition check); Telugu/Hindi voice polish
24 – 30	End-to-end rehearsal on the iQOO 15; on-device vs. cloud latency benchmark; demo recording; pitch

Why this plan is credible: the risky plumbing — local inference, OCR, voice I/O, APK pipeline — is already working in the prototype. The battle adds features to a proven foundation, not hope to a blank repo.

Zero marginal cost is the unlock

The wedge

Medicine explanation + triage support for ASHA workers in Telangana — a concrete, reachable first user group we can test with directly.

The scale

~1M ASHA workers × multiple home visits a day = tens of millions of assisted interactions per month — with **no cloud bill standing between this and national scale**, because every query runs on hardware people already own.

Beyond ASHA

The same offline engine serves elders living alone, low-literacy patients, and any rural household: an AI health assistant that works precisely because it *doesn't* need the internet.

Safety, honestly

Sahayak is a **screening and explanation aid — never a diagnosis**. Dosage is read from the label, never invented. Danger signs always trigger "refer immediately." Every answer ends by pointing to a doctor or pharmacist. All health data stays on the device.

07 · WHY WE'LL DELIVER

Judged on execution? Good — that's what we brought.

Between registering and submitting this form, we shipped a working offline-AI Android app: engine abstraction, two complete camera/voice modules, model hot-swapping, CI, and an installable release. That is our answer to every "but can they build it in 30 hours?" — we already build at that pace, and the phone-first tooling to keep that pace through the Red Light windows is inside the app itself.

Sahayak — because the last mile of healthcare shouldn't depend on the nearest cell tower.

Repo, installable APK, and prototype video linked on the cover page. We're ready to build.